A Simple Five-Bit Quantum Random Digit Generator

A Transparent Rejection-Sampling Protocol with Extreme-Run Recycling

Eric C. Thompson
Independent Researcher, Fulshear, Texas, USA
ORCID: 0009-0001-4127-1742
Email: ethompson06@gmail.com

Abstract

Quantum random number generators provide physical randomness by converting intrinsically probabilistic measurement outcomes into usable digital values. This white paper presents a simple five-bit protocol for generating uniformly distributed decimal digits from an ideal 50/50 quantum source. Five independent binary outcomes produce $2^5 = 32$ possible sequences. By rejecting the two extreme runs, 00000 and 11111, exactly 30 admissible sequences remain, allowing a direct uniform mapping onto the ten decimal digits.

The main contribution of this note is not entropy-optimal extraction, but transparency: the protocol is easy to inspect, easy to implement, and easy to explain to nontechnical users. This makes it especially relevant for trust-critical public-facing applications where visible fairness may matter more than maximum entropy efficiency. A small efficiency improvement is also introduced. Instead of discarding a rejected extreme run entirely, the rejected all-A or all-B sequence is recycled as a single fair starting bit for the next five-bit candidate block. Under the assumption of independent fair input bits, this carry-forward rule preserves exact decimal uniformity while improving the expected raw-bit cost from $16/3 \approx 5.333$ bits per digit to $79/15 \approx 5.267$ bits per digit.

The paper also discusses practical implementation using a beam-splitter-style quantum randomness source, limitations under detector imbalance, and the proper cryptographic positioning of the method. The protocol can serve as a transparent entropy source or educational QRNG digitization scheme, but cryptographic deployment would still require standard health testing, calibration, and extraction.

Contents

1 Introduction

Random numbers are essential in cryptography, simulations, randomized algorithms, gaming, lotteries, secure communications, and many forms of digital authentication. Classical pseudo-random number generators are deterministic algorithms: given the same state, they produce the same output. This distinction between deterministic computation and physical random-digit generation goes back at least to von Neumann's discussion of random digits and correction procedures [1]. Quantum random number generators, by contrast, use measurement outcomes from quantum systems as a physical source of indeterminacy.

A deterministic mathematical formula can transform, expand, or disguise randomness that is already present in a seed, but it cannot create fresh entropy by itself. If the internal state and algorithm are known, the output is fixed. For true physical randomness, some observation of an entropy-bearing natural process is required. In a QRNG, that role is played by the quantum measurement event: the digitization protocol does not manufacture randomness, but converts physical measurement outcomes into usable digital symbols.

This paper describes a simple method for converting ideal quantum binary outcomes into uniform decimal digits. The method groups five binary outcomes into a candidate block. Since five bits give 32 possible outcomes, and since 30 is divisible by 10, rejecting two selected outcomes leaves exactly three admissible sequences per decimal digit.

The simplest exclusion choice is to reject the two extreme runs:

$$00000 \quad \text{and} \quad 11111.$$

Equivalently, if the two detector outcomes are labeled A and B, the rejected strings are

$$\text{AAAAA} \qquad \text{and} \qquad \text{BBBBB}.$$

The remaining 30 outcomes can then be mapped uniformly to digits $0, \ldots, 9$.

The protocol is deliberately modest. It is not intended to beat modern entropy extractors or commercial QRNG post-processing systems. Its value is simplicity, auditability, and a clean decimal interface. The protocol is useful as a transparent construction, an educational demonstration, or a lightweight digit-generation module when the input source is already well-characterized.

2 Background

2.1 Quantum binary outcomes

A common conceptual QRNG setup uses a single photon incident on a balanced optical element, such as a 50/50 beam splitter [2, 3]; optical and quantum-information approaches to randomness generation are part of the broader quantum-cryptographic technology landscape [4]. If the system is ideal, each trial produces one of two possible detector outcomes with equal probability:

$$\mathbb{P}(\text{A}) = \mathbb{P}(\text{B}) = \frac{1}{2}.$$

The detector outcome can be encoded as a bit:

$$\text{A} \leftrightarrow 0, \qquad \text{B} \leftrightarrow 1.$$

Repeating the experiment produces a binary stream.

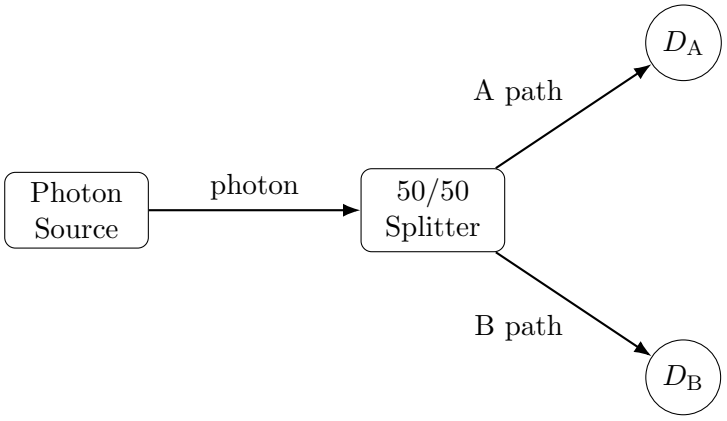

Figure 1: Conceptual QRNG architecture. A balanced two-outcome quantum measurement produces binary outcomes, which are then grouped into five-bit candidate blocks.

2.2 From binary outcomes to decimal digits

A decimal digit requires at least

$$\log_2(10) \approx 3.322$$

bits of entropy in the information-theoretic limit. In practice, fixed-size binary blocks do not always map evenly into ten outcomes. For example, four bits give 16 possible values, which cannot be evenly divided among ten digits without rejection. Five bits give 32 possible values, and rejecting exactly two leaves 30, which divides evenly by 10.

3 The Five-Bit Extreme-Sequence Exclusion Protocol

3.1 Protocol definition

Definition 1 (Five-bit extreme-sequence exclusion protocol)**.** *Assume a sequence of independent fair binary outcomes*

$$X_1, X_2, X_3, \ldots \in \{0, 1\},$$

with

$$\mathbb{P}(X_i = 0) = \mathbb{P}(X_i = 1) = \frac{1}{2}.$$

The protocol proceeds as follows:

1. *Read five outcomes and form a five-bit block*

$$x = x_1 x_2 x_3 x_4 x_5.$$

2. *Interpret the block as an integer*

$$v(x) = 16x_1 + 8x_2 + 4x_3 + 2x_4 + x_5,$$

   so that $v \in \{0, \ldots, 31\}$.

3. *Reject the two extreme strings:*

$$00000 \quad (v = 0), \qquad 11111 \quad (v = 31).$$

4. *For every accepted value* $v \in \{1, \ldots, 30\}$*, output*

$$d = v \bmod 10.$$

3.2 Uniformity

Proposition 1 (Uniform decimal output)**.** *For independent fair input bits, the five-bit extreme-sequence exclusion protocol outputs each decimal digit* $d \in \{0, \ldots, 9\}$ *with probability* $1/10$*.*

Proof. There are $2^5 = 32$ possible five-bit strings. Under the fair-bit assumption, each string has probability $1/32$. The protocol rejects exactly two strings, leaving 30 accepted strings. The accepted values are

$$v = 1, 2, \ldots, 30.$$

Modulo 10, these values form three complete residue classes:

$$1, \ldots, 10, \qquad 11, \ldots, 20, \qquad 21, \ldots, 30.$$

Thus each decimal digit occurs exactly three times among the 30 accepted strings. Conditioned on acceptance, each accepted string has equal probability. Therefore each digit has probability

$$\frac{3}{30} = \frac{1}{10}.$$

□

4 Efficiency of the Basic Protocol

Each five-bit candidate block is accepted unless it is 00000 or 11111. Therefore the acceptance probability is

$$p = \frac{30}{32} = \frac{15}{16}.$$

The rejection probability is

$$q = \frac{2}{32} = \frac{1}{16}.$$

The expected number of five-bit blocks required per accepted digit is

$$\frac{1}{p} = \frac{16}{15}.$$

Thus the expected raw-bit cost per decimal digit is

$$5 \cdot \frac{16}{15} = \frac{16}{3} \approx 5.333.$$

The entropy efficiency relative to the decimal entropy limit is

$$\frac{\log_2(10)}{16/3} \approx 0.623.$$

This is not entropy optimal, but it is substantially simpler than large-block arithmetic coding, nonuniform random generation, or cryptographic extraction methods designed to approach the entropy limit [5].

5 Extreme-Run Carry-Forward Recycling

5.1 Motivation

In the basic protocol, the rejected strings 00000 and 11111 are discarded entirely. However, conditioned on the event that a rejection occurred, the rejected string is equally likely to be 00000 or 11111 under the fair-bit assumption. Therefore the identity of the rejected extreme run contains one fair bit of information.

This motivates a small recycling rule:

$$00000 \longrightarrow \text{carry forward one } 0,$$

$$11111 \longrightarrow \text{carry forward one } 1.$$

The carried bit becomes the first bit of the next five-bit candidate block. Only four fresh quantum outcomes are then needed to complete the next candidate block.

5.2 Recycling protocol

Definition 2 (Extreme-run carry-forward protocol)**.** *The carry-forward protocol is identical to the basic five-bit protocol, except when a rejected extreme string occurs:*

1. *If the rejected string is* 00000*, carry forward one* 0 *as the first bit of the next candidate block.*
2. *If the rejected string is* 11111*, carry forward one* 1 *as the first bit of the next candidate block.*
3. *Draw four fresh independent quantum bits to complete the next candidate block.*
4. *Apply the same rejection and modulo-*10 *mapping rule to the resulting five-bit candidate block.*

5.3 Uniformity with recycling

Theorem 1 (Recycling preserves uniformity)**.** *Under the assumption of independent fair input bits, the extreme-run carry-forward protocol preserves exact uniformity of the decimal output.*

Proof. In the basic five-bit protocol, the two rejected strings are 00000 and 11111. Since each five-bit string is equally likely,

$$\mathbb{P}(00000) = \mathbb{P}(11111) = \frac{1}{32}.$$

Conditioned on rejection,

$$\mathbb{P}(00000 \mid \text{rejection}) = \mathbb{P}(11111 \mid \text{rejection}) = \frac{1}{2}.$$

Therefore the carried bit is itself a fair bit.

The next candidate block consists of this fair carried bit together with four fresh independent fair bits. Hence the next candidate block is uniformly distributed over all 32 possible five-bit strings. The original uniformity proof applies unchanged: after excluding 00000 and 11111, the 30 accepted strings map three-to-one onto the ten decimal digits. Therefore the output remains exactly uniform. □

5.4 Efficiency with recycling

The first candidate block after an accepted output requires five fresh raw bits. If it is rejected, the next candidate block already has one carried bit and requires only four fresh bits.

The probability of rejection for each candidate block remains

$$q = \frac{1}{16},$$

and the probability of acceptance remains

$$p = \frac{15}{16}.$$

The expected number of rejections before the next acceptance is

$$\frac{q}{p} = \frac{1/16}{15/16} = \frac{1}{15}.$$

Therefore the expected raw-bit cost per output digit is

$$5 + 4\left(\frac{1}{15}\right) = \frac{79}{15} \approx 5.267.$$

The absolute saving compared with the basic protocol is

$$\frac{16}{3} - \frac{79}{15} = \frac{1}{15} \approx 0.0667$$

raw bits per digit. Equivalently, over 30 generated decimal digits, the basic protocol uses 160 raw bits in expectation, while the carry-forward protocol uses 158 raw bits in expectation. The improvement is therefore small but exact:

$$\frac{160 - 158}{160} = 1.25\%.$$

Remark 1. *The efficiency gain is modest, but the rule is conceptually useful. It demonstrates that the two rejected extreme states are not entirely worthless; under ideal fairness, their identity contains exactly one reusable fair bit.*

6 Bias, Calibration, and Practical Limits

The uniformity proofs above assume independent fair binary outcomes. A real QRNG may have detector imbalance, optical asymmetry, electronic timing bias, dead-time effects, dark counts, afterpulsing, or correlations between trials. The five-bit exclusion rule removes modulo bias under ideal fair-bit assumptions, but it does not by itself remove physical source bias.

Suppose

$$\mathbb{P}(0) = r, \qquad \mathbb{P}(1) = 1 - r.$$

Then the rejected strings have probabilities

$$\mathbb{P}(00000) = r^5, \qquad \mathbb{P}(11111) = (1-r)^5.$$

Conditioned on rejection, the probability that the carried bit is 0 becomes

$$\mathbb{P}(0 \mid \text{rejection}) = \frac{r^5}{r^5 + (1-r)^5}.$$

This equals $1/2$ only when $r = 1/2$. Thus, if the physical source is biased, the carry-forward bit can inherit an amplified bias from the extreme-run condition.

For this reason, practical use should follow standard entropy-source validation principles and should include [6]:

1. detector calibration;
2. monitoring of the empirical A/B balance;
3. tests for autocorrelation and drift;
4. rejection-rate monitoring;
5. independent health checks;
6. and, for cryptographic applications, a standard randomness extractor or deterministic random bit generator seeded by validated entropy.

More generally, practical QRNGs typically require entropy evaluation, health testing, and post-processing or extraction before cryptographic use [7, 6].

The protocol should therefore be understood as a transparent digitization layer, not as a complete cryptographic QRNG certification procedure.

7 Implementation Architecture

A practical implementation can be built from standard QRNG components.

7.1 Quantum source

The binary outcomes may be produced by a physical system with two nominally equal outcomes, such as:

1. a photon incident on a balanced beam splitter;
2. phase-noise sampling from a quantum optical source;

3. vacuum-fluctuation measurement with thresholding, as in high-bandwidth optical QRNG implementations [8, 9, 10];

4. or another validated two-outcome quantum process.

The beam-splitter picture is the easiest to explain, but it is not the only possible implementation.

7.2 Detection and timing

The detector system records which output channel fires first or which channel corresponds to the accepted quantum measurement event. The two channels are labeled

$$\mathrm{A} \equiv 0, \qquad \mathrm{B} \equiv 1.$$

In an optical implementation, avalanche photodiodes or superconducting nanowire detectors could be used, depending on cost, speed, and target performance.

7.3 Digitization logic

A microcontroller, FPGA, or embedded processor can implement the protocol:

1. collect five candidate bits;

2. reject 00000 and 11111;

3. optionally recycle the rejected extreme as a carried bit;

4. compute v mod 10 for accepted values $v = 1, \ldots, 30$;

5. output a decimal digit.

The modulo operation is trivial for modern hardware, and the full mapping table can also be stored as a small lookup table.

8 Cryptographic Positioning

A QRNG can improve cryptographic systems by supplying high-quality entropy for:

1. key generation;

2. session secrets;

3. salts;

4. nonces;

5. initialization vectors;

6. seed material for deterministic random bit generators;

7. and one-time-pad systems where key distribution is feasible.

However, a QRNG does not by itself make classical public-key cryptography safe against large-scale quantum computers. Algorithms such as RSA and elliptic-curve cryptography are vulnerable because quantum algorithms can attack the underlying factoring and discrete-log structures [11]. Better randomness cannot remove that structural vulnerability.

The proper post-quantum role of a QRNG is therefore supportive rather than substitutive. It can provide strong entropy to post-quantum cryptographic systems, but it does not replace post-quantum algorithms. In particular:

1. RSA and elliptic-curve systems should be migrated to post-quantum key-exchange and signature schemes where long-term quantum resistance is required; NIST's finalized post-quantum standards include ML-KEM for key establishment and ML-DSA and SLH-DSA for digital signatures [12].

2. Symmetric cryptography can be strengthened by using sufficiently large keys, such as 256-bit keys, because quantum search attacks provide a quadratic rather than exponential speedup [13].

3. QRNGs can provide valuable entropy for both classical and post-quantum systems, but the entropy source must be validated and monitored.

Thus, the five-bit protocol is best described as a transparent QRNG digitization method, not as a standalone defense against quantum attacks.

9 Novelty and Scope

The mathematical family to which this protocol belongs is rejection sampling: a larger set of equally likely source outcomes is reduced to a smaller target set by rejecting surplus outcomes. That general idea is well known in random generation and sampling theory [1, 5]. The present contribution is narrower:

1. a simple five-bit construction for uniform decimal digits;

2. exclusion of the two visually intuitive extreme runs;

3. a complete mapping table from accepted five-bit strings to decimal digits;

4. a practical QRNG-oriented device description;

5. and an extreme-run carry-forward rule that slightly improves efficiency while preserving exact uniformity under ideal fair-bit assumptions.

The protocol should be distinguished from three established areas. First, QRNG hardware and entropy-source validation are well developed in both academic and commercial systems. Second, rejection sampling and modulo-bias avoidance are standard techniques for converting uniform source bits into bounded outcomes. Third, randomness extraction and randomness recycling are established methods for improving statistical quality and entropy efficiency.

The contribution of the present note is narrower: it gives a compact five-bit decimal digitization rule designed for public explainability. The rule uses the arithmetic coincidence $2^5 - 2 = 30 = 3 \times 10$, rejects the two visually transparent extreme runs, and maps the remaining strings exactly three-to-one onto decimal digits. The optional carry-forward rule reuses the identity of a rejected extreme run as one fair seed bit under ideal source assumptions. This recycling step is related in spirit to

broader randomness-recycling methods in exact and online sampling, although here it is specialized to the two rejected extreme runs of the five-bit decimal protocol [14]. This combination is intended as a transparent QRNG digit interface rather than an entropy-optimal extractor.

9.1 Low-latency optical implementation

Although the protocol is not entropy-optimal, it is naturally suited to low-latency implementation. A beam-splitter QRNG can generate binary outcomes from optical measurement events, and the digitization layer requires only five binary outcomes per candidate decimal digit. The post-processing step is minimal: reject two special strings and apply a modulo-10 map or lookup table.

The relevant advantage is therefore not that the protocol reaches a fundamental speed limit, but that it combines a fast physical randomness source with a very small processing window. In practical devices, the realized output rate will be limited by photon source rate, detector recovery time, timing electronics, health testing, and output bandwidth. Still, the five-bit block size gives the protocol a low-latency character: each candidate digit is decided after only five physical binary samples, or after four new samples when the carry-forward recycling rule is triggered.

This makes the scheme attractive for applications where immediate, explainable decimal outputs are more important than asymptotic entropy efficiency.

9.2 Trust-critical public-facing randomness

A major motivation for the protocol is not maximum entropy efficiency, but public explainability. In some applications, the users of a random number generator are not cryptographers, physicists, or engineers. They may be ordinary participants who simply need to trust that the outcome was not secretly manipulated.

Examples include public lotteries, educational demonstrations, online games, gambling systems, raffles, transparent audit systems, and consumer-facing random selection tools. In such settings, a more efficient extractor may be mathematically superior but harder for a general audience to understand. By contrast, the five-bit protocol has a simple public story: a fair quantum source produces five binary outcomes; the two extreme runs are excluded; the remaining thirty outcomes divide evenly into ten decimal digits.

This makes the protocol useful where visible fairness and explainability are part of the design goal. The protocol should not be understood as replacing formal certification, regulatory compliance, cryptographic extraction, or independent auditing. Rather, its advantage is that the fairness mechanism can be communicated in a compact and intuitive way.

The protocol is therefore not presented as an entropy-optimal extractor or a new foundation for cryptographic randomness. Its strength is that it is easy to audit, explain, implement, and verify.

10 Conclusion

The five-bit extreme-sequence exclusion protocol provides a simple method for generating uniform decimal digits from an ideal two-outcome quantum random source. Five fair bits produce 32 possible strings. Rejecting the two extreme strings, 00000 and 11111, leaves 30 accepted strings, which map evenly onto the ten decimal digits.

The basic protocol requires

$$\frac{16}{3} \approx 5.333$$

raw quantum bits per decimal digit in expectation. The carry-forward improvement recycles the identity of a rejected extreme run as one fair starting bit for the next candidate block. This lowers the expected cost to

$$\frac{79}{15} \approx 5.267$$

raw bits per digit, a 1.25% improvement.

The resulting method is not optimized for maximum entropy efficiency, but it is unusually transparent. That transparency may be valuable for educational QRNGs, audit-friendly random digit services, public lotteries, regulated gaming environments, lightweight devices, and demonstrations of quantum randomness. These are settings where users may need a simple explanation of why the output is fair.

For serious cryptographic deployment, the protocol should be embedded within a broader validated QRNG architecture that includes calibration, health testing, source monitoring, and extraction.

A Mapping Table

The following table uses A = 0 and B = 1. The integer value v is computed from the five-bit string. The two extreme sequences are rejected. Every accepted value is mapped to a decimal digit using v mod 10.

Sequence	Bits	v	Output
AAAAA	00000	0	REJECT
AAAAB	00001	1	1
AAABA	00010	2	2
AAABB	00011	3	3
AABAA	00100	4	4
AABAB	00101	5	5
AABBA	00110	6	6
AABBB	00111	7	7
ABAAA	01000	8	8
ABAAB	01001	9	9
ABABA	01010	10	0
ABABB	01011	11	1
ABBAA	01100	12	2
ABBAB	01101	13	3
ABBBA	01110	14	4
ABBBB	01111	15	5
BAAAA	10000	16	6
BAAAB	10001	17	7
BAABA	10010	18	8
BAABB	10011	19	9
BABAA	10100	20	0
BABAB	10101	21	1
BABBA	10110	22	2
BABBB	10111	23	3
BBAAA	11000	24	4

Sequence	Bits	v	Output
BBAAB	11001	25	5
BBABA	11010	26	6
BBABB	11011	27	7
BBBAA	11100	28	8
BBBAB	11101	29	9
BBBBA	11110	30	0
BBBBB	11111	31	REJECT

B Pseudocode

B.1 Basic protocol

```
while true:
    block = read_5_quantum_bits()
    v = integer_value(block)

    if v == 0 or v == 31:
        continue

    digit = v mod 10
    output(digit)
```

B.2 Carry-forward protocol

```
carry_exists = false
carry_bit = none

while true:
    if carry_exists:
        block = [carry_bit] + read_4_quantum_bits()
        carry_exists = false
    else:
        block = read_5_quantum_bits()

    v = integer_value(block)

    if v == 0:
        carry_bit = 0
        carry_exists = true
        continue

    if v == 31:
        carry_bit = 1
        carry_exists = true
        continue
```

```
digit = v mod 10
output(digit)
```

References

[1] J. von Neumann, "Various techniques used in connection with random digits," in *Monte Carlo Method*, National Bureau of Standards Applied Mathematics Series, vol. 12, pp. 36–38, 1951.

[2] ID Quantique, "Quantis: Quantum Random Number Generator – QRNG," application/solution note, 2017.

[3] M. Herrero-Collantes and J. C. Garcia-Escartin, "Quantum random number generators," *Reviews of Modern Physics*, vol. 89, 015004, 2017.

[4] S. Pirandola et al., "Advances in quantum cryptography," *Advances in Optics and Photonics*, vol. 12, no. 4, pp. 1012–1236, 2020.

[5] D. E. Knuth and A. C. Yao, "The complexity of nonuniform random number generation," in *Algorithms and Complexity: New Directions and Recent Results*, Academic Press, pp. 357–428, 1976.

[6] National Institute of Standards and Technology, *Recommendation for the Entropy Sources Used for Random Bit Generation*, NIST Special Publication 800-90B, 2018.

[7] X. Ma, F. Xu, H. Xu, X. Tan, B. Qi, and H.-K. Lo, "Postprocessing for quantum random-number generators: Entropy evaluation and randomness extraction," *Physical Review A*, vol. 87, 062327, 2013.

[8] T. Symul, S. M. Assad, and P. K. Lam, "Real time demonstration of high bitrate quantum random number generation with coherent laser light," *Applied Physics Letters*, vol. 98, 231103, 2011.

[9] J. Y. Haw, S. M. Assad, A. M. Lance, N. H. Y. Ng, V. Sharma, P. K. Lam, and T. Symul, "Maximization of extractable randomness in a quantum random-number generator," *Physical Review Applied*, vol. 3, 054004, 2015.

[10] Australian National University, "ANU QRNG: Quantum random numbers," public quantum random number service.

[11] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," in *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*, pp. 124–134, 1994.

[12] National Institute of Standards and Technology, "NIST releases first 3 finalized post-quantum encryption standards," U.S. Department of Commerce, August 13, 2024.

[13] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, pp. 212–219, 1996.

[14] T. L. Draper and F. A. Saad, "Efficient Online Random Sampling via Randomness Recycling," arXiv:2505.18879, 2025.